

3

Laboratory 3

3	Foreground object segmentation	27
3.1	Main objectives	
3.2	Foreground object segmentation	
3.3	Methods based on frame buffer	
3.4	Approximation of mean and median (so-called sigma-delta)	
3.5	Updating policy	
3.6	OpenCV – GMM/MOG	
3.7	OpenCV – KNN	
3.8	Example results of the algorithm for foreground object segmentation	
3.9	Using a neural network to segment foreground objects	

3. Foreground object segmentation

3.1 Main objectives

- familiarisation with the issue of foreground object segmentation and the problems associated with it,
- implementation of simple background modelling algorithms based on a sample buffer – analysis of their advantages and disadvantages,
- implementation of running mean and median approximation algorithms – analysis of their advantages and disadvantages,
- learning about the methods available in *OpenCV* – the Gaussian Mixture Model (also called Mixture of Gaussians) and the method based on the KNN (K-Nearest Neighbours) algorithm,
- familiarisation with the architecture of a neural network and an example application.

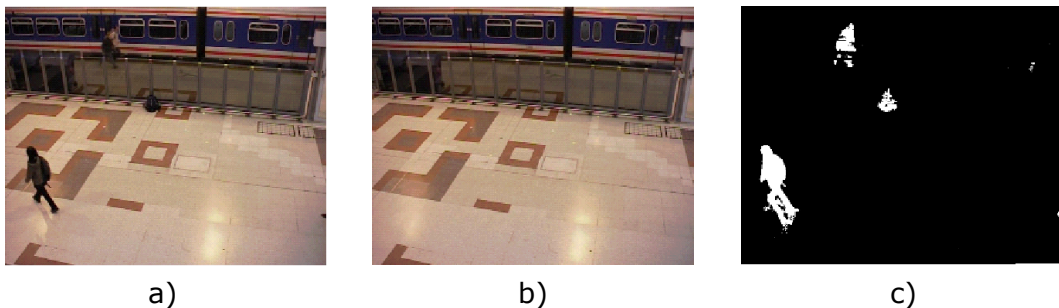


Figure 3.1: Przykład segmentacji obiektów pierwszoplanowych z wykorzystaniem modelowania tła. a) bieżąca ramka, b) model tła, c) maska obiektów pierwszoplanowych. Źródło: sekwencja [PETS 2006](#).

3.2 Foreground object segmentation

Foreground object segmentation

Segmentation consists of extracting the objects of interest from the image. This does not have to mean *classification* (e.g. *car*, *pedestrian*); often it is sufficient to know *where* in the image the object is located. The notion of a *foreground object* depends on the application (e.g. in surveillance: people and vehicles).

Background model

In the simplest view, this is an image of an “empty scene”, i.e. without foreground objects. In practice, in many methods the background model is implicit (it is not a single image) and cannot be directly displayed (e.g. GMM, ViBE, PBAS).

Background model initialisation

The model has to be initialised at the start of the algorithm. Most often, the first frame (or the first N frames in buffer-based methods) is used as the starting point. Assumption: in the initialisation phase there are no foreground objects. If objects are present, the model starts with an error, which in the literature is referred to as *bootstrap*. More robust approaches analyse changes over time (and sometimes also spatial coherence), assuming that the background is visible for most of the time.

Background modelling (generation)

A static “background image” works only under strictly controlled conditions. In typical scenes, the background changes (lighting, time of day) or elements of the scene are rearranged (e.g. a chair). Therefore, the background model should adapt; this update process is called background modelling/generation. Lack of adaptation (or adaptation that is too slow) leads to false detections, including *ghosts*.

Pitfalls in background modelling

There is no single method that works well for all scenes. Typical problematic situations include:

- noise and compression artifacts (MJPEG, H.264/265, MPEG),
- camera jitter,
- camera automation (white balance, exposure),
- lighting changes: slow (time of day) and sudden,
- objects present during initialisation,
- moving background elements,
- camouflage (object similar to background),
- objects rearranged in the background.

Ghost (in the background model)

Example: a car stops in a parking lot and after some time becomes incorporated into the background. When it drives away, the “empty spot” may be detected as an object – this is precisely the *ghost* (an object present in the model but not in the current frame).

Update policy

The model update can be:

- **conservative** – only pixels classified as background are updated,
- **liberal** – all pixels are updated.

A conservative policy may “freeze” an error (pixels are never corrected), while a liberal one promotes blending objects into the background and the creation of *ghosts* or streaks.

Shadows

The shadow cast by an object usually meets the criteria of the “foreground”, but from the point of view of further analysis it is a disturbance (it changes shape, connects objects). Therefore, a shadow is often treated as a separate class: neither background nor foreground object.

Example

Figure 3.2 shows segmentation using a background model. Note the shadow under the feet of the person on the left and the person behind the barrier (barely visible in the current frame).

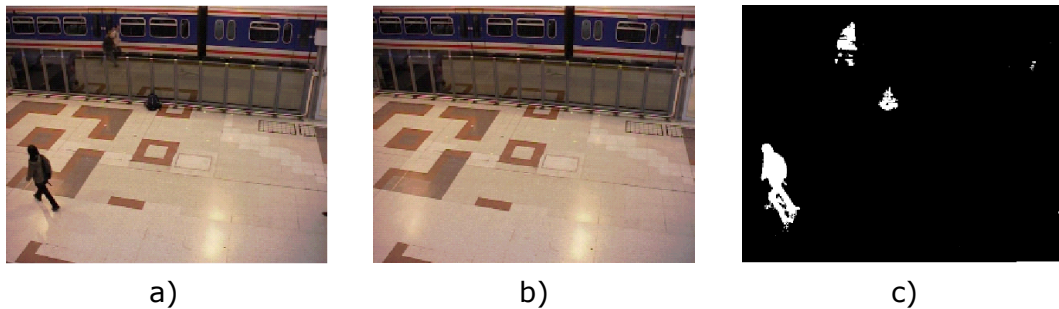


Figure 3.2: Example of foreground object segmentation using background modelling. a) current frame, b) background model, c) foreground object mask. Source: sequence [PETS 2006](#).

3.3 Methods based on frame buffer

The exercise will present two methods: the buffer mean and the buffer median. The buffer size will be assumed to be $N = 60$ samples. In each iteration of the algorithm it is necessary to remove the last frame from the buffer, add the current one (the buffer works on the principle of FIFO queue) and calculate the mean or median from the buffer.

1. First, declare a buffer of size $N \times YY \times XX$ (`np.zeros` function), where YY and XX are the height and width of the frame from the *pedestrians* sequence.

```
BUF = np.zeros((YY,XX,N),np.uint8)
```

The buffer counter (e.g. `iN`) should also be initialised with the value 0. The `iN` parameter acts as a pointer: it indicates the position in the buffer at which the oldest frame should be removed and replaced with the current frame.

2. The buffer should be handled as follows: in a loop, at the position indicated by `iN`, the current greyscale frame should be stored.

```
BUF[:, :, iN] = IG;
```

Next, the counter `iN` should be incremented and checked to see whether it has reached the buffer size (N). If it has, it must be reset to 0.

3. The calculation of the mean or median is performed using the `mean` or `median` function from the *numpy* library. The dimension (axis) for which the value is to be computed is passed as a parameter (remember that indexing starts from zero). To ensure everything works correctly, the result should be converted to `uint8`.

4. Finally, we perform background subtraction, i.e. we subtract the model from the current frame, and then binarise the result – analogously to the case of the difference between neighbouring frames. Additionally, we apply median filtering to the object mask and/or use morphological operations.
5. Using the code created in the previous exercise, compare the behaviour of the method using the mean and the median. Record the value of the $F1$ score for both cases.

R Computing the median may take a longer while.

Consider why the median works better. Test the behaviour on other sequences – in particular on the *office* sequence. Observe the phenomena of smearing and the silhouette blending into the background, as well as the appearance of *ghosts*.

Exercise 3.1 Implement the algorithms (mean and median) based on the above description. ■

3.4 Approximation of mean and median (so-called sigma-delta)

Using a sample buffer requires significant memory resources. The mean can be updated in a simple way (it is worth considering how to update the mean in the buffer without recalculating all elements). In the case of the median, the situation is more difficult: there are no equally fast algorithms for computing it, although it is not necessary to sort all elements at each iteration. Therefore, methods that do not require a buffer are used more often. In the case of approximating the mean, the following relation is used:

$$BG_N = \alpha I_N + (1 - \alpha) BG_{N-1} \quad (3.1)$$

where: I_N – current frame, BG_N – background model, α – weight parameter (typically 0.01-0.05, although the value depends on the specific application).

The approximation of the median is obtained using the relation:

$$BG_N = \begin{cases} BG_{N-1} + 1, & \text{if } BG_{N-1} < I_N, \\ BG_{N-1} - 1, & \text{if } BG_{N-1} > I_N, \\ BG_{N-1}, & \text{otherwise.} \end{cases} \quad (3.2)$$

Exercise 3.2 Implement both methods.

1. Take the first frame of the sequence as the first background model. Note the value of the $F1$ indicator for the two methods (set the *alpha* parameter to 0.01). Observe the running time.
2. Experiment with the value of the *alpha* parameter. See how the parameter value affects the background model.

R For the running average method, it is crucial that **the background model is stored in a floating-point format** (*float64*).

In formula 3.1, avoiding the use of a loop over the entire image is straightforward. For relation 3.2 this is also possible, but it requires a moment of thought. It is worth consulting the *numpy* documentation for this purpose. Please also note that in Python it is possible to perform arithmetic operations on boolean values.

3.5 Updating policy

So far we have followed a liberal update policy – we have updated everything. We will now try to use a conservative approach – we update only those pixels that have been classified as **background**.

Exercise 3.3 1. For the selected method, implement a conservative update approach. Check its operation.

R Simply remember the previous object mask and use it appropriately in the update procedure.

2. Note the value of the F1 indicator and the background model. Are there any errors in it?

3.6 OpenCV – GMM/MOG

In the OpenCV library, one of the most popular methods for foreground object segmentation is available: Gaussian Mixture Models (GMM), also known as Mixture of Gaussians (MoG) – both names are used interchangeably in the literature. In short, the scene is modelled using several Gaussian distributions (e.g. of mean brightness/colour and standard deviation). Each distribution is also described by a weight that reflects how frequently it has been observed in the scene (i.e. its probability of occurrence). The distributions with the highest weights represent the background. Segmentation consists of computing the distance between the current pixel observation and each of the distributions. If the pixel is similar to a distribution recognised as background, it is classified as background; otherwise, it is considered an object. The background model is also updated in a manner similar to equation 3.1. Readers who are interested in more details are referred to the literature, for example to the work by [Stauffer and Grimson](#).

Exercise 3.4 This method is an example of a multivariant algorithm – the background model has several possible representations (variants).

1. Use the `BackgroundSubtractorMOG2` class by creating an object with `createBackgroundSubtractorMOG2`. In a loop, for each image, call the `apply()` method.
2. Experiment with the `history` and `varThreshold` parameters and disable shadow detection. In the `apply()` method you can also set the learning rate – *learningRate*.
3. Compute the *F1* measure (for the method without shadow detection). Observe how the method works. Note that it is not possible to “display” the background model.

R The OpenCV package provides a version of GMM that is slightly different from the original one – among other things, it includes a shadow detection module and a dynamically changing number of Gaussian components.

3.7 OpenCV – KNN

The second method available in OpenCV is KNN algorithm (`BackgroundSubtractorKNN`).

Exercise 3.5 Please run the KNN method and evaluate it. ■

3.8 Example results of the algorithm for foreground object segmentation

In order to better verify the individual steps of the proposed methods for foreground object segmentation, example results for selected image frames will be presented.

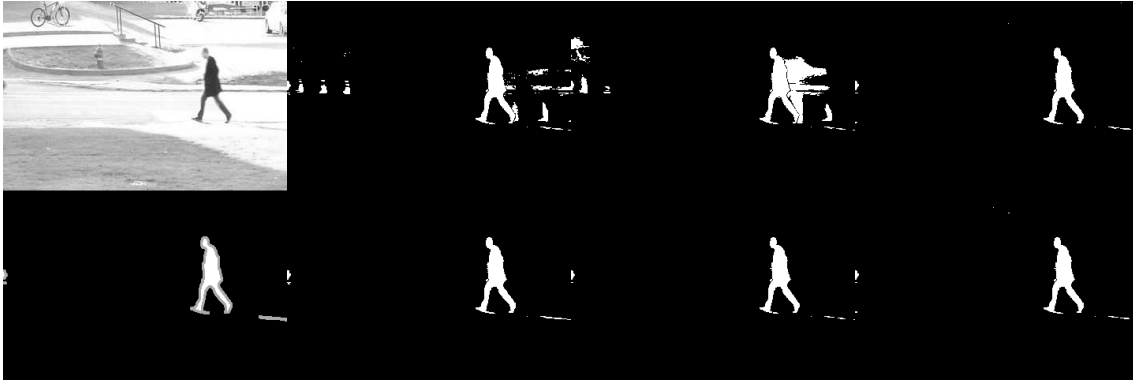


Figure 3.3: Example of the result of the different versions of the algorithm for foreground object segmentation for the *pedestrians* dataset. The frame index is 600. The first column from the left is the input image together with the reference image. The first row is for algorithms using the mean, the second row uses the median. Sequentially from the second column the result of the algorithm is presented: liberal update policy, then the function approximation with a liberal update policy and the function approximation with a conservative update policy.



Figure 3.4: Example of the result of the different versions of the algorithm for foreground object segmentation for the *highway* dataset. The frame index is 1200. The first column from the left is the input image together with the reference image. The first row is for algorithms using the mean, the second row uses the median. Sequentially from the second column the result of the algorithm is presented: liberal update policy, then the function approximation with a liberal update policy and the function approximation with a conservative update policy.

Exercise completion

In order to have the exercise accepted, after completing all the tasks in this laboratory,

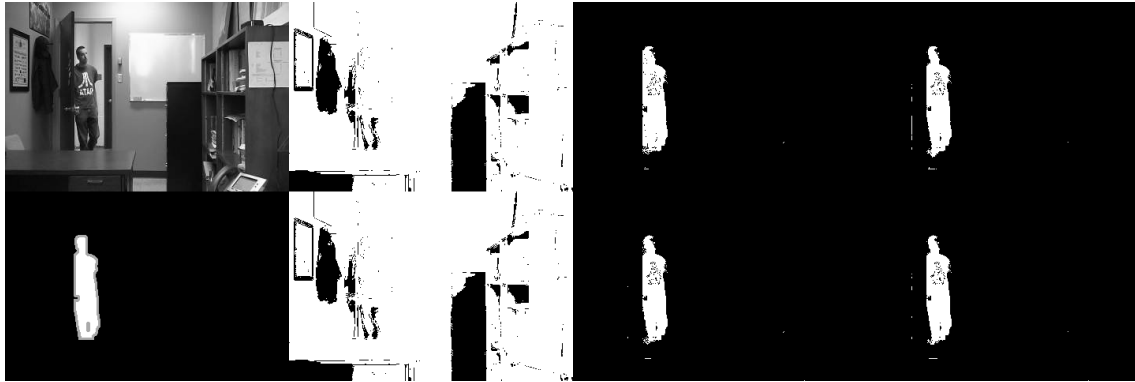


Figure 3.5: Example of the result of the different versions of the algorithm for foreground object segmentation for the *office* dataset. The frame index is 600. The first column from the left is the input image together with the reference image. The first row is for algorithms using the mean, the second row uses the median. Sequentially from the second column the result of the algorithm is presented: liberal update policy, then the function approximation with a liberal update policy and the function approximation with a conservative update policy.

report your readiness to the teacher. Once it has been approved by the teacher, place the script with the `.py` or `.ipynb` extension in the appropriate location in the course resources on UPeL.

3.9 Using a neural network to segment foreground objects

Neural network architecture-based methods can also be used to segment foreground objects. An example of this is the work¹. The proposed approach is to use a convolutional neural network called BSUV-Net (Background Subtraction Unseen Videos Net). The architecture is based on a U-net structure with residual connections, which is divided into an encoder and a decoder. The input to network consists of the current frame and two background frames captured at different time scales along with their semantic segmentation maps. The output is a mask containing only foreground objects.

You can find details about this neural network in the article <https://arxiv.org/abs/1907.11371>. The architecture with sample data and trained network models is made available on Github <https://github.com/ozantezcan/BSUV-Net-inference>.

Exercise 3.6 Non-obligatory assignment. Run the BSUV-Net convolutional neural network.

- Download all the files for the neural network from the UPeL platform.
- The `pedestrians` database will be used, but appropriately scaled to the size of the network input and converted to video sequences.
- Open `infer_config_manualBG.py` file and modify the paths to:
 - in class `SemanticSegmentation` specify the absolute path to the segmentation folder – variable `root_path`,
 - in class `BSUVNet` specify the path to the network model `BSUV-Net-2.0.mdl` in the `trained_models` folder – `model_path`,
 - in class `BSUVNet` specify the path to an image that contains only the background – the first image in the set `pedestrians` – variable `empty_bg_path`.
- In the `inference.py` file specify the path to the network input, i.e. the `pedestrians` video sequence – variable `inp_path` and a path to where the network output is to be stored (path with the file name and file extension) – variable `out_path`.

¹Tezcan, Ozan and Ishwar, Prakash and Konrad, Janusz; BSUV-Net: A Fully-Convolutional Neural Network for Background Subtraction of Unseen Videos, <https://arxiv.org/abs/1907.11371>.